

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..8

STREAMING

 Copy as Markdown

Streaming

Some features need the server to push data to the app the moment it changes: chat, multiplayer games, live dashboards. Streaming methods cover this: an endpoint method that returns or receives a Dart `Stream`, transmitted over a shared WebSocket connection that Serverpod manages for you.

For a real-world example, check out [Pixorama](#). It's a multi-user drawing experience that showcases Serverpod's real-time capabilities. Its complete source code is available.

Streaming methods

When an endpoint method is defined with `Stream` instead of `Future` as the return type, or includes `Stream` as a method parameter, it is recognized as a streaming method.

Define a streaming method

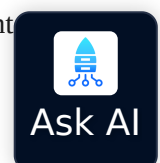
Streaming methods are defined by using the `Stream` type as either the return value or a parameter.

Following is an example of a streaming method that echoes back any message:

```
class ExampleEndpoint extends Endpoint {  
  Stream echoStream(Session session, Stream stream) async* {  
    await for (var message in stream) {  
      yield message;  
    }  
  }  
}
```

The `Stream` can also be given a type argument, such as `Stream<String>`. The type carries into the generated client, so the compiler checks the messages you send and receive.

The streaming method above can then be called from the client like this:



```
var inStream = StreamController();
var outStream = client.example.echoStream(inStream.stream);
outStream.listen((message) {
  print('Received message: $message');
});

inStream.add('Hello');
inStream.add(42);

// This will print
// Received message: Hello
// Received message: 42
```

In the example above, the `echoStream` method reads each message from its stream parameter and sends it back on its return stream. These streams have no type argument, so they are dynamic and can carry any type that Serverpod can serialize.

Lifecycle of a streaming method

Each time the client calls a streaming method, a new `Session` is created for the call, and it closes automatically when the call is over. If the WebSocket connection is lost, all streaming methods are closed on both the server and the client.

How long the call stays alive depends on the method's shape:

- A method returning a `Stream` is kept alive until the client cancels its subscription or the method returns.
- A method returning a `Future` (with `Stream` parameters) is kept alive until the method returns.

A stream parameter closes when the client closes the stream or the server cancels its subscription to it, and all stream parameters close when the call ends.

Authentication

Authentication is integrated into streaming method calls. When a client initiates a streaming method, the server automatically authenticates the session.

Authentication is validated when the stream is first established, using the authentication data stored in the `Session` object. If a user's authentication is revoked, the stream is closed and an exception is thrown.

For more details on handling revoked authentication, refer to the section on [handling revoked authentication](#).

WebSocket ping interval

The server sends periodic ping messages on open streaming connections to keep them alive. The interval between pings is configurable and defaults to 30 seconds.

If you deploy behind a load balancer or proxy with a shorter idle timeout (for example, 15-20 seconds), you may need to lower the ping interval so connections are not closed. Set the

`SERVERPOD_WEBSOCKET_PING_INTERVAL` environment variable to the desired interval in seconds, or configure `websocketPingInterval` in your config file. See the [Configuration reference](#).

Error handling

A streaming call can fail in three ways:

- Errors your code raises travel over the stream as [serializable exceptions](#).
- When the server refuses to open the stream, the call fails with one of the [ServerpodClientHttpException](#) subclasses that a regular call uses. For example, a caller who is not signed in gets [ServerpodClientUnauthorized](#).
- Failures in the connection itself throw a [MethodStreamException](#) subtype.

If an exception is thrown on a stream, the stream is closed with an exception. If the thrown exception is serializable, it is serialized and delivered over the stream before the stream closes, in both directions: stream parameters can pass exceptions to the server, and return streams can pass exceptions to the client.

Define the exception in a model file, like any [serializable exception](#):

```
exception: CountdownException
fields:
  message: String
```

Throw it from the streaming method, and catch it on the stream in your app:

```
class ExampleEndpoint extends Endpoint {
  Stream<int> countdown(Session session) async* {
    yield 3;
    yield 2;
    throw CountdownException(message: 'Countdown aborted');
  }
}
```

```
var stream = client.example.countdown();
stream.listen((number) {
  print('Countdown: $number');
}, onError: (error) {
  if (error is CountdownException) {
    print('Countdown failed: ${error.message}');
  }
});
```

In the other direction, add the error to a stream parameter on the client, and catch it typed on the server around the `await for`:

```
Future<String> reportErrors(Session session, Stream<int> values) async {  
  try {  
    await for (final _ in values) {}  
    return 'completed without error';  
  } on CountdownException catch (e) {  
    return 'caught: ${e.message}';  
  }  
}
```

```
var controller = StreamController<int>();  
var result = client.example.reportErrors(controller.stream);  
controller.addError(CountdownException(message: 'Error from client'));  
await controller.close();  
print(await result); // caught: Error from client
```

Connection-level exceptions

When the failure is in the connection rather than your code, the client throws a subtype of `MethodStreamException`:

Exception	When it is thrown
<code>WebSocketConnectException</code>	The WebSocket connection to the server could not be opened.
<code>ConnectionAttemptTimedOutException</code>	Opening the connection did not complete in time.
<code>WebSocketListenException</code>	Listening on the WebSocket failed after it was opened.
<code>WebSocketClosedException</code>	The WebSocket closed while the stream was in use.
<code>ConnectionClosedException</code>	The stream connection closed, for example when authentication was revoked.
<code>MethodStreamIdleTimeoutException</code>	The stream was idle past the client's idle timeout.

Catch `MethodStreamException` to handle them as one group, or match specific subtypes when the reaction differs.

Example: live updates for a filtered query

A common real-time need is to push updates for one record or filter, for example the messages in a single chat room. Combine a streaming method with [server events](#): the streaming method subscribes the client to a channel scoped to that filter, and whatever changes the data posts to the same channel.

```
class ChatEndpoint extends Endpoint {  
  // The client subscribes to live messages for one room.  
  Stream<ChatMessage> watchRoom(Session session, int roomId) async* {  
    yield* session.messages.createStream<ChatMessage>('room_${roomId}');  
  }  
  
  // Posting a message stores it and broadcasts it to everyone watching the room.  
  Future<void> postToRoom(Session session, int roomId, String text) async {  
    var message = ChatMessage(roomId: roomId, text: text);  
    await ChatMessage.db.insertRow(session, message);  
    await session.messages.postMessage('room_${roomId}', message);  
  }  
}
```

On the client, listen to the returned stream and call `postToRoom` to publish:

```
var messages = client.chat.watchRoom(roomId);  
messages.listen((message) {  
  print('Room ${message.roomId}: ${message.text}');  
});  
  
await client.chat.postToRoom(roomId, 'Hello, room!');
```

Because the channel name includes the `roomId`, each client receives updates only for the room it is watching. The same pattern works for any filter: scope the channel by the id or query you care about, and post to it whenever the data changes. To fan the updates out across multiple server instances, enable Redis (see [Message scope](#)).